



Chapter 2 – Software Processes and Models

Software Engineering, by Ian Sommerville 10th edition, Pearson Education Limited, 2016. and

Software Engineering – A Practitioner's Approach 8th edition Roger Pressman McGraw Hill Education (2015).

Topics covered



- ✧ Software process
- ✧ Software process models

The software process



- ✧ A structured set of activities required to develop a software system.
- ✧ Many different software processes, but all involve:
 - **Specification** – defining what the system should do;
 - **Design and implementation** – defining the organization of the system and implementing the system;
 - **Validation** – checking that it does what the customer wants;
 - **Evolution** – changing the system in response to changing customer needs.
- ✧ A **software process model** is an abstract representation of a process.
 - It presents a description of a process from some particular perspective.

Software process descriptions



✧ Process descriptions may also include:

- **Products**, which are the **outcomes of a process activity**;
- **Roles**, which reflect the **responsibilities of the people involved** in the process;
- **Pre- and post-conditions**, which are **statements that are true before and after a process activity** has been enacted or a product produced.

Plan-driven and agile processes



✧ Plan-driven processes:

- Activities are planned in advance, and progress is measured against this plan.
- Separate and distinct phases of specification and development
- Example: **The waterfall model**

✧ Agile processes:

- Planning is incremental, and it is
- Easier to change the process to reflect changing customer requirements.
- Example: **Extreme Programming (XP) model**

✧ In practice, most practical processes include elements of both plan-driven and agile approaches.

Software process models



✧ Incremental development

- Specification, development and validation are **interleaved**.
- May be plan-driven or agile.

✧ Integration and configuration

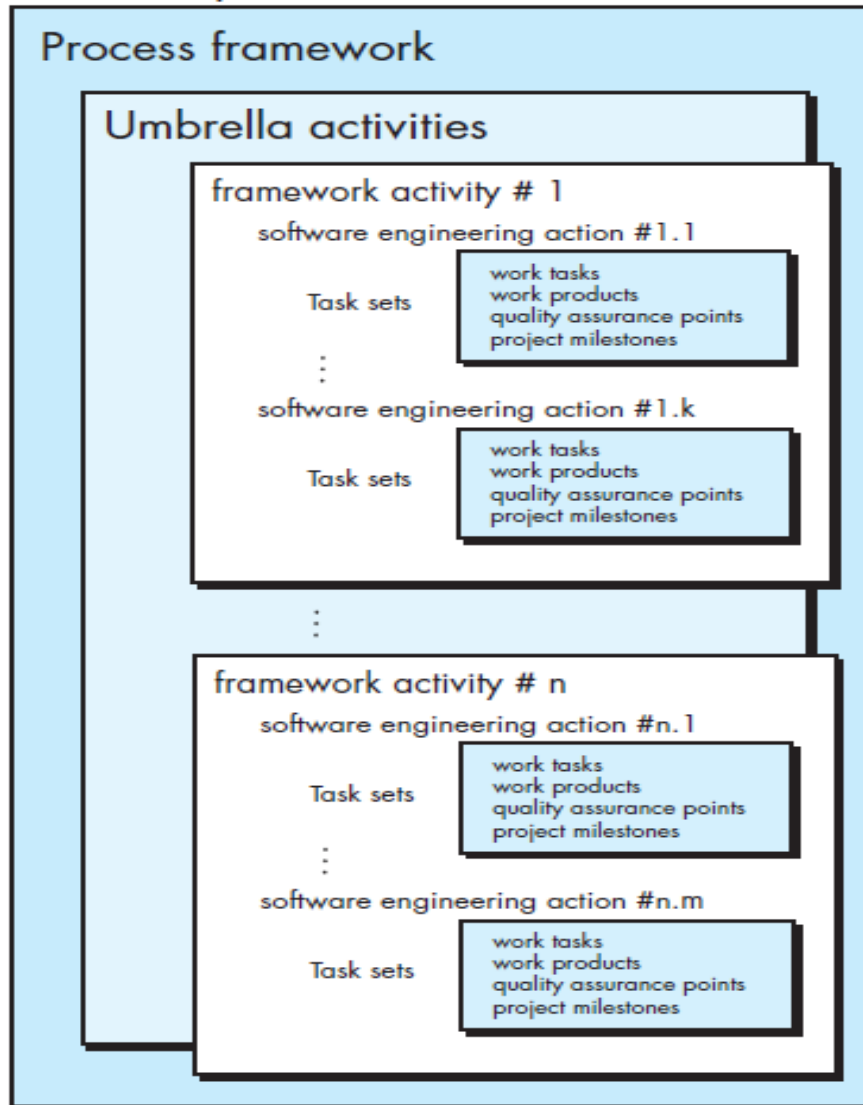
- The system is **assembled from existing configurable** components. May be plan-driven or agile.

✧ In practice, most large systems are developed using a process that incorporates elements from all of these models.

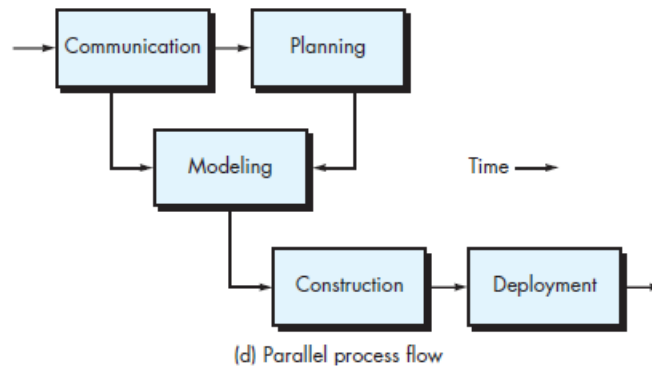
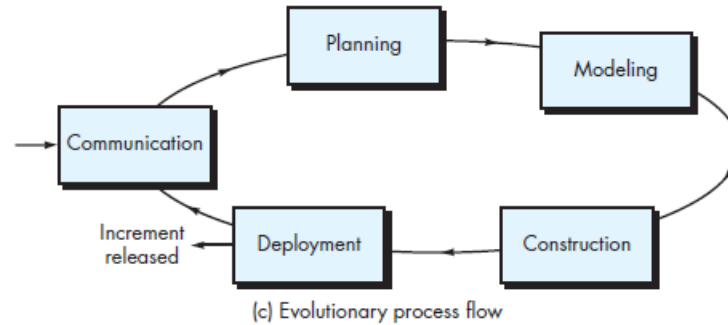
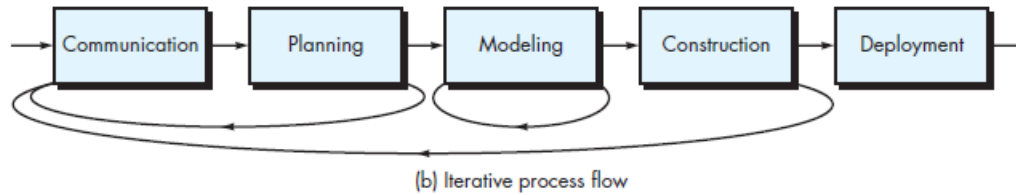
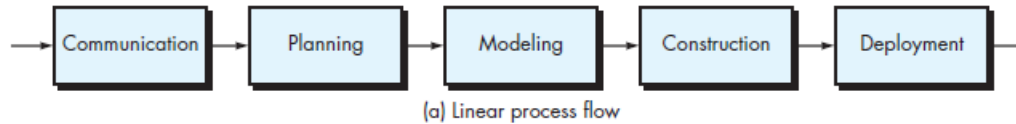


Software process models

A Generic Process Model



Process flow



Prescriptive / Plan-driven Models



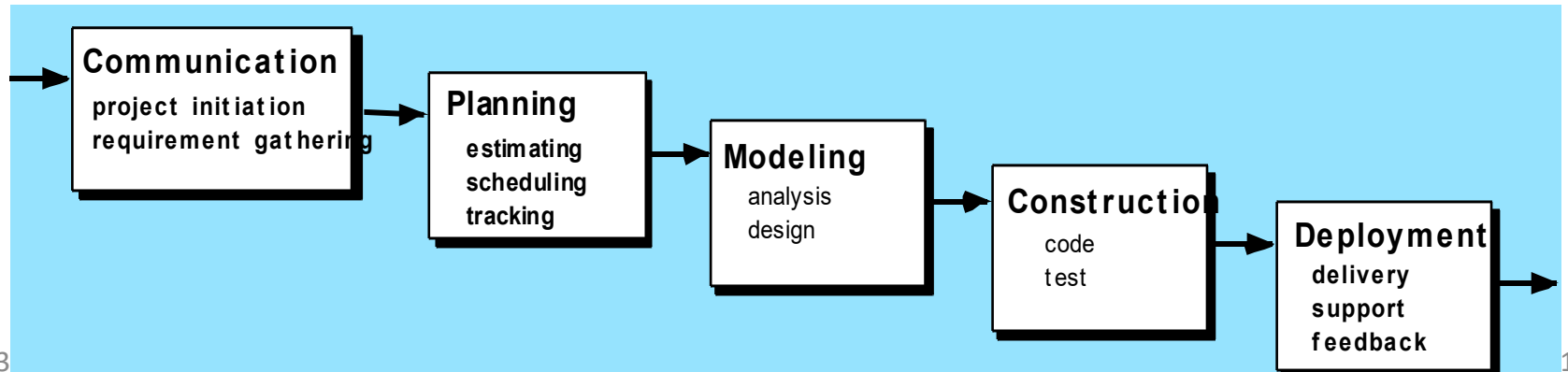
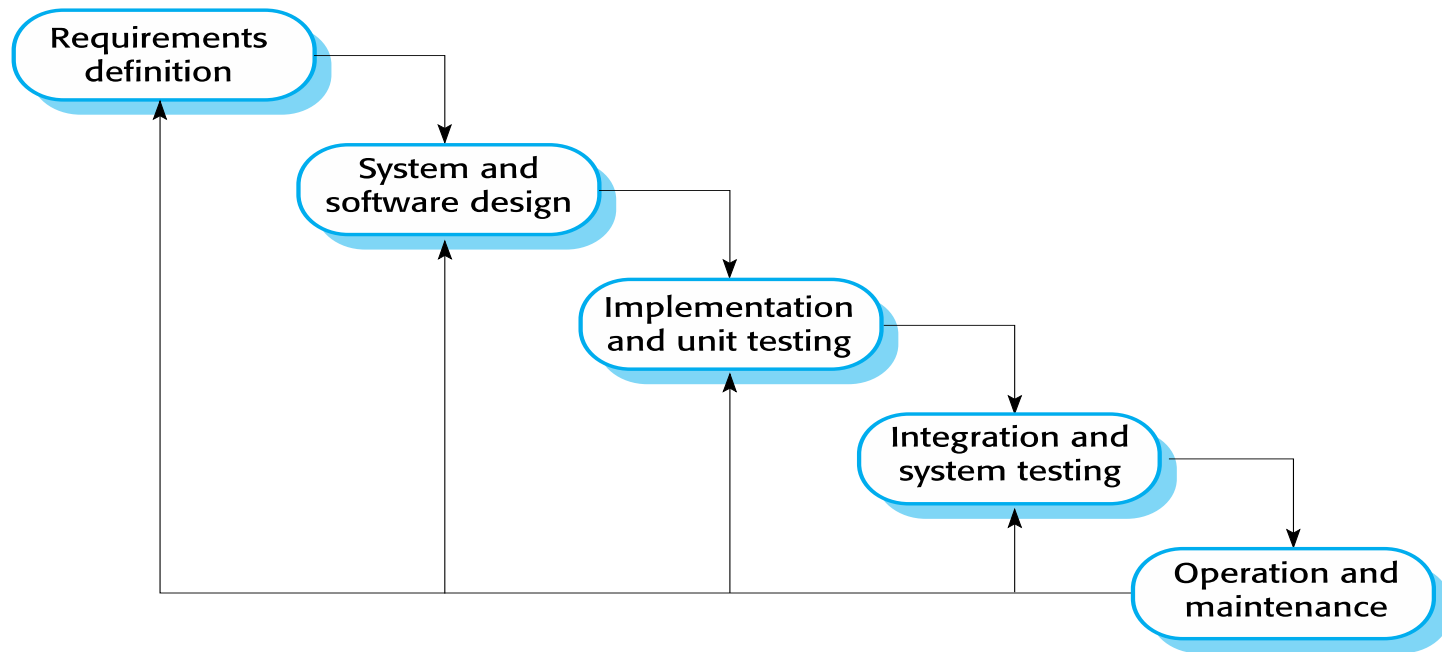
Arguments:

- ✧ Prescriptive process models advocate an orderly approach to software engineering

That leads to a few questions ...

- ✧ If prescriptive process models strive for structure and order, *are they inappropriate for a software world that thrives on change?*
- ✧ Yet, if we reject traditional process models (and the order they imply) and replace them with something less structured, *do we make it impossible to achieve coordination and coherence in software work?*

The waterfall model



Waterfall model phases



✧ Separate identified phases in the waterfall model:

- Requirements analysis and definition
- System and software design
- Implementation and unit testing
- Integration and system testing
- Operation and maintenance

✧ The main drawback:

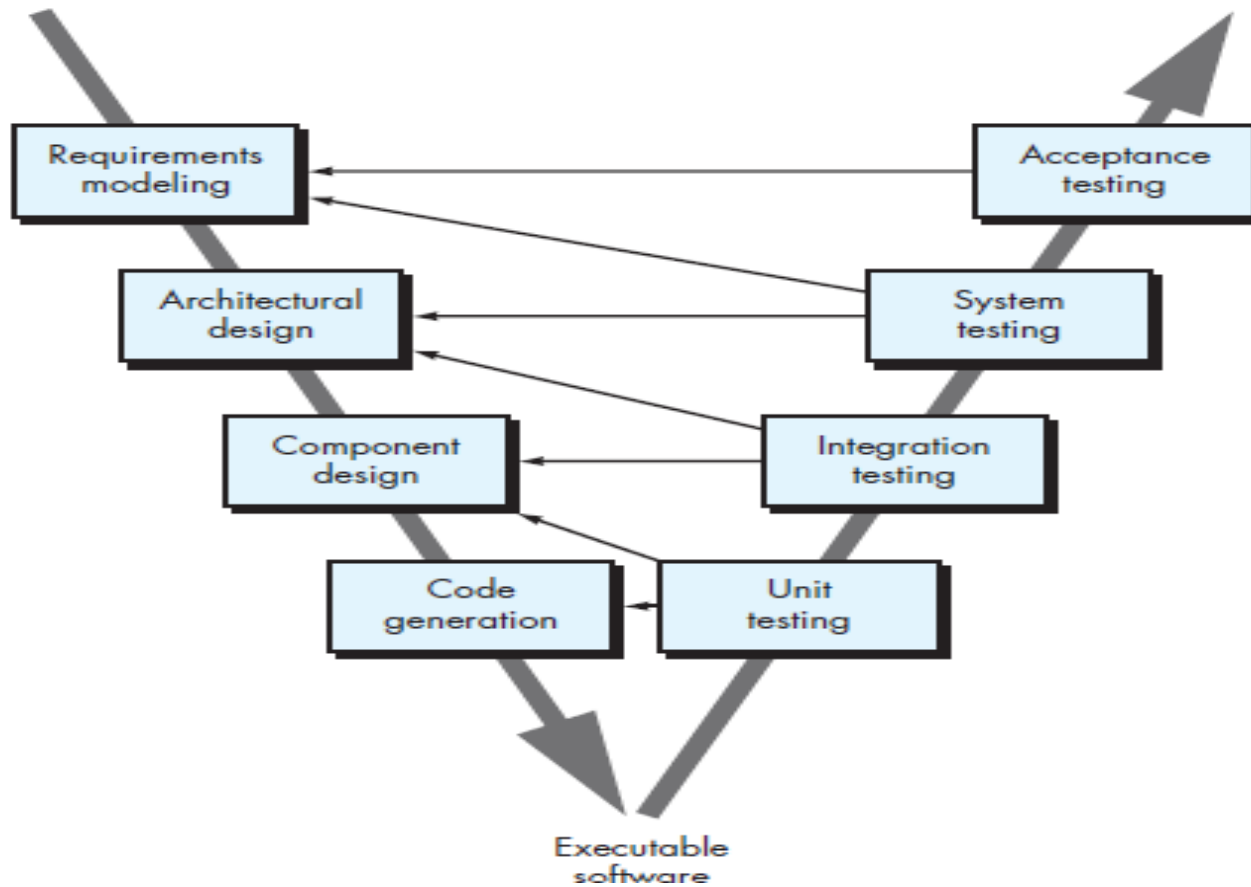
- Difficulty of accommodating change after the process is underway.
 - In principle, a phase has to be complete before moving onto the next phase.

Waterfall model problems

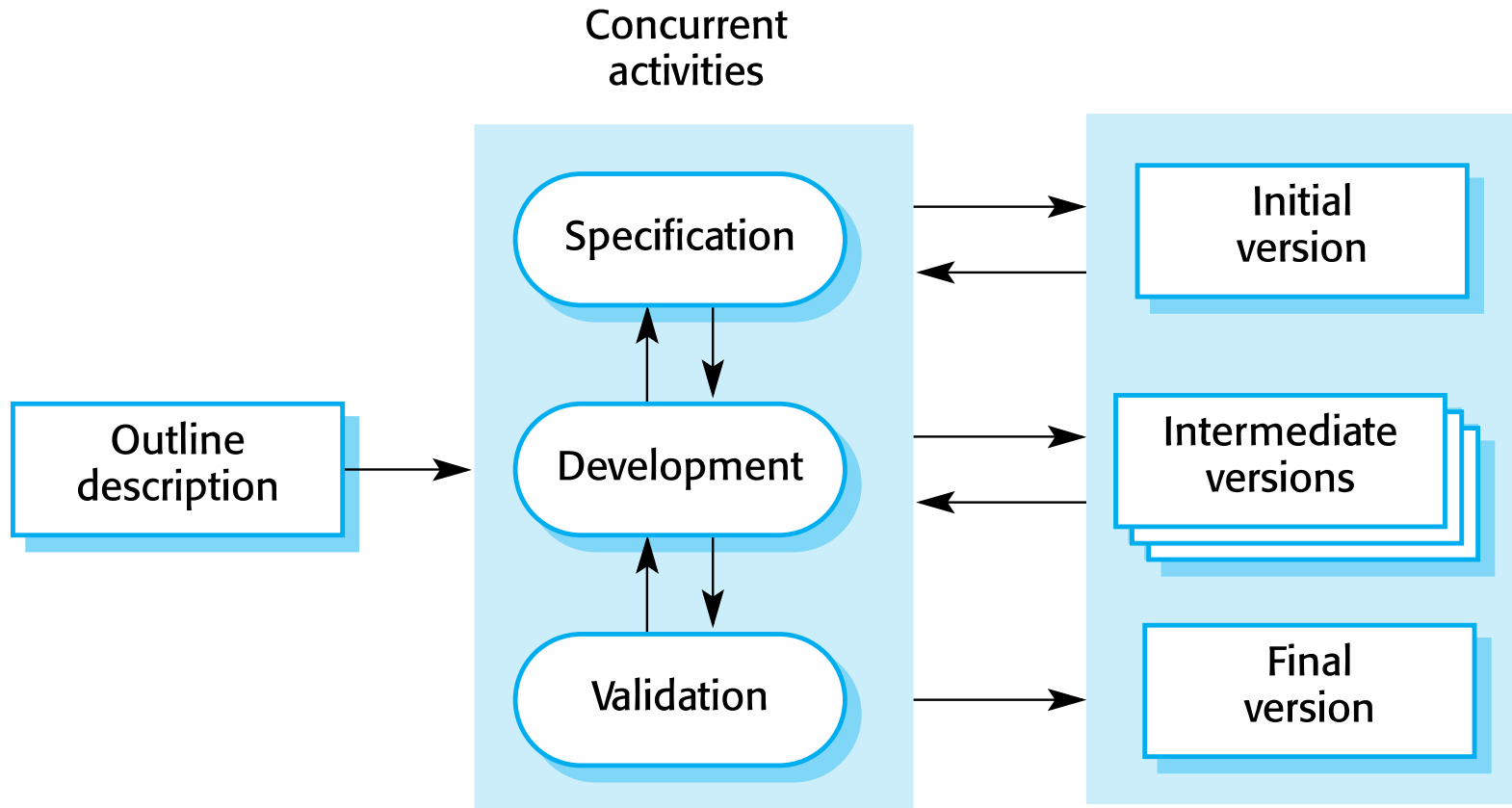


- ✧ **Inflexible** partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
- Only appropriate when the requirements are well-understood.
 - Mostly used for large systems engineering projects

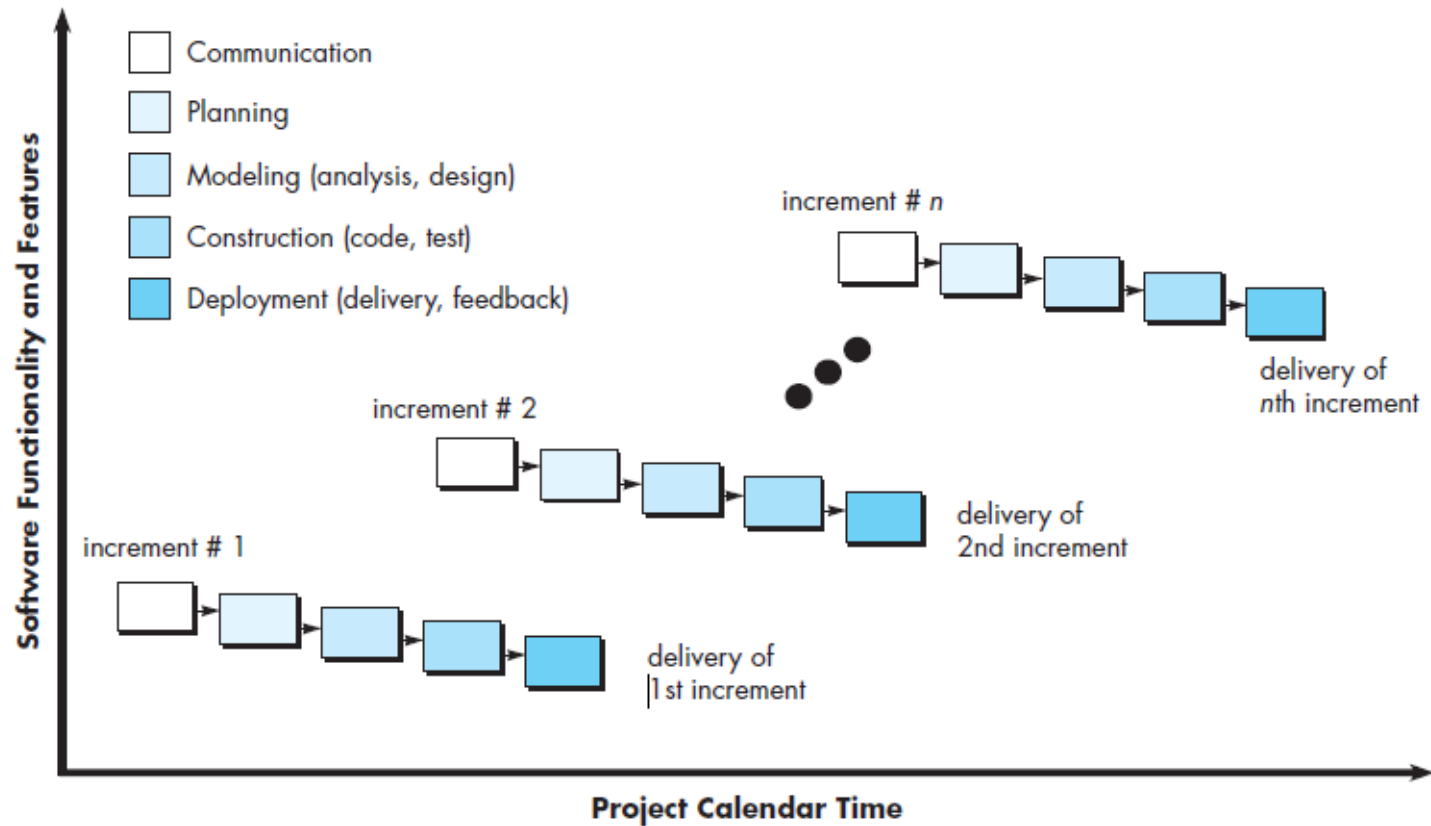
The V-Model



The incremental model



The incremental model



Incremental delivery



- ✧ Rather than deliver the system as a single delivery, the development and delivery are broken down into increments, with each increment delivering part of the required functionality.
- ✧ User requirements are prioritised, and the highest priority requirements are included in early increments.
- ✧ Once the development of an increment is started, the requirements are **frozen**, though requirements for later increments can continue to evolve.

Incremental delivery advantages



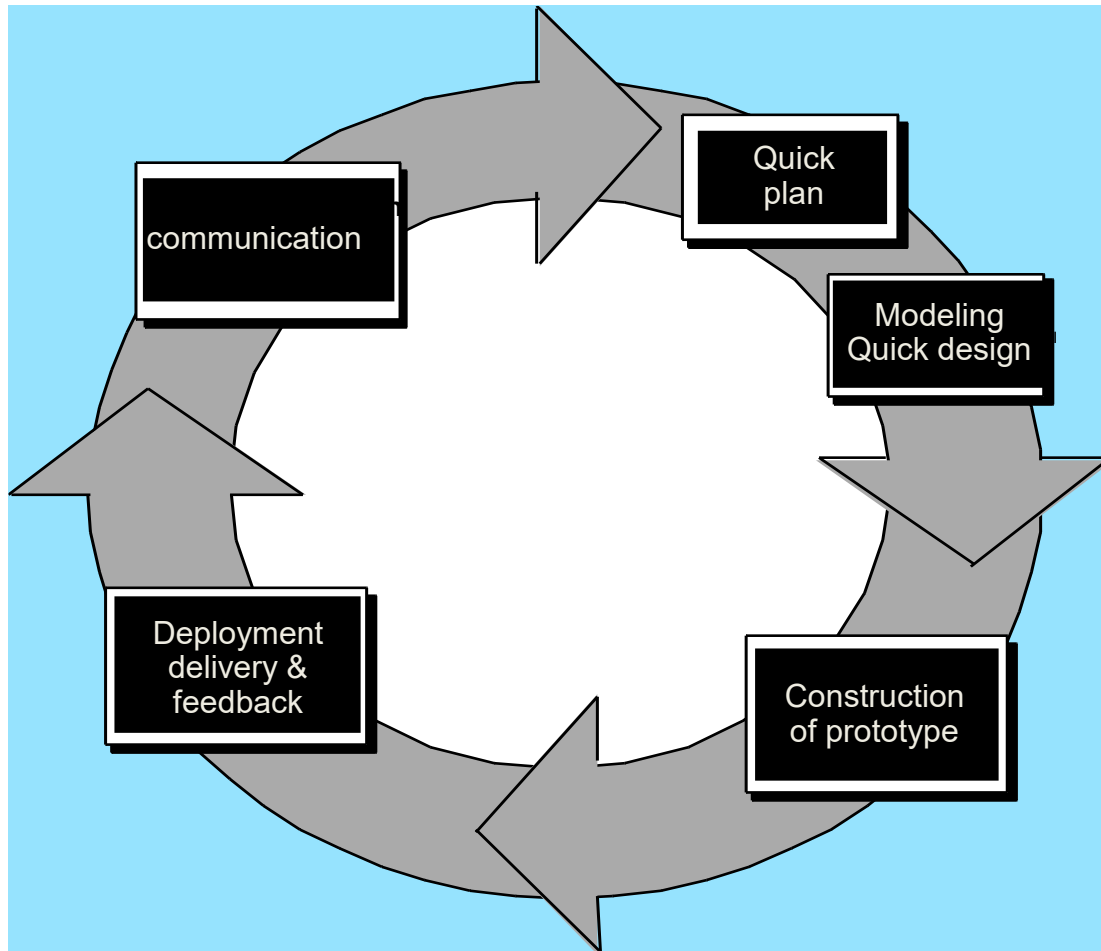
- ✧ Customer value can be delivered with each increment so system **functionality is available earlier**.
- ✧ **Early increments act as a prototype** to help elicit requirements for later increments.
- ✧ **Lower risk of overall project failure**.

Incremental delivery problems

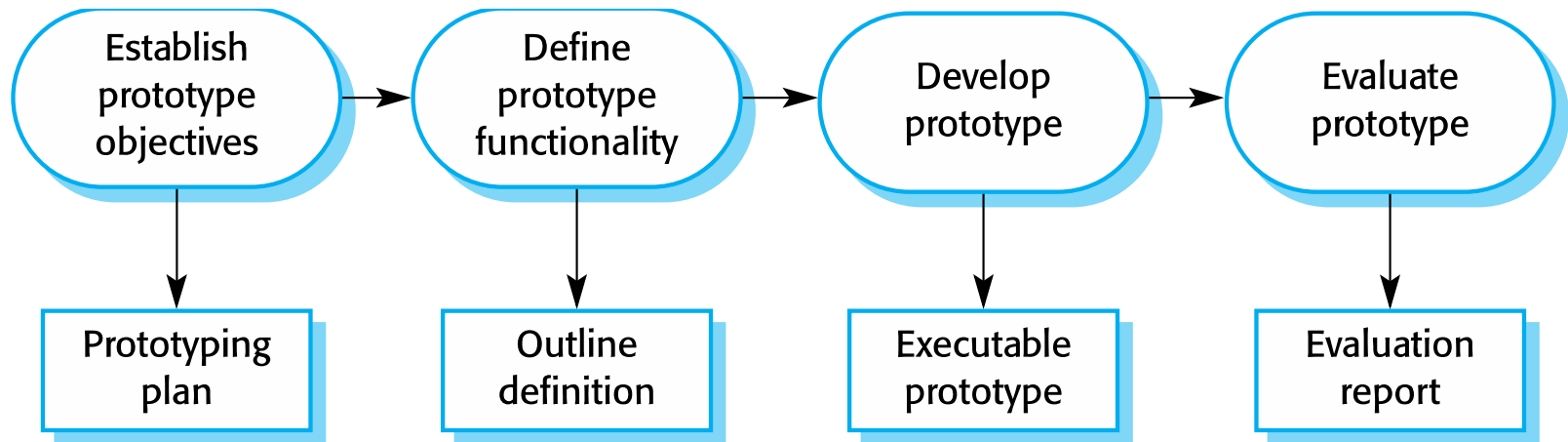


- ✧ As requirements are not defined in detail until an increment is to be implemented, it can be hard to identify common facilities that are needed by all increments.
- ✧ System structure tends to degrade as new increments are added
- ✧ Scope Creep via Feedback
- ✧ Difficulty Slicing Work
- ✧ Lack of Overall Planning

Evolutionary Models: Prototyping



The process of prototype development



Software prototyping



- ✧ A **prototype** is an initial version of a system used to demonstrate concepts and try out design options.
- ✧ **A prototype can be used in:**
 - The **requirements** engineering process to help with requirements elicitation and validation;
 - In **design** processes to explore options and develop a UI design;
 - In the **testing** process, run back-to-back tests.

Benefits of prototyping



- ✧ Improved system usability.
- ✧ A closer match to users' real needs.
- ✧ Improved design quality.
- ✧ Improved maintainability.
- ✧ Reduced development effort.

Prototype development



✧ May involve:

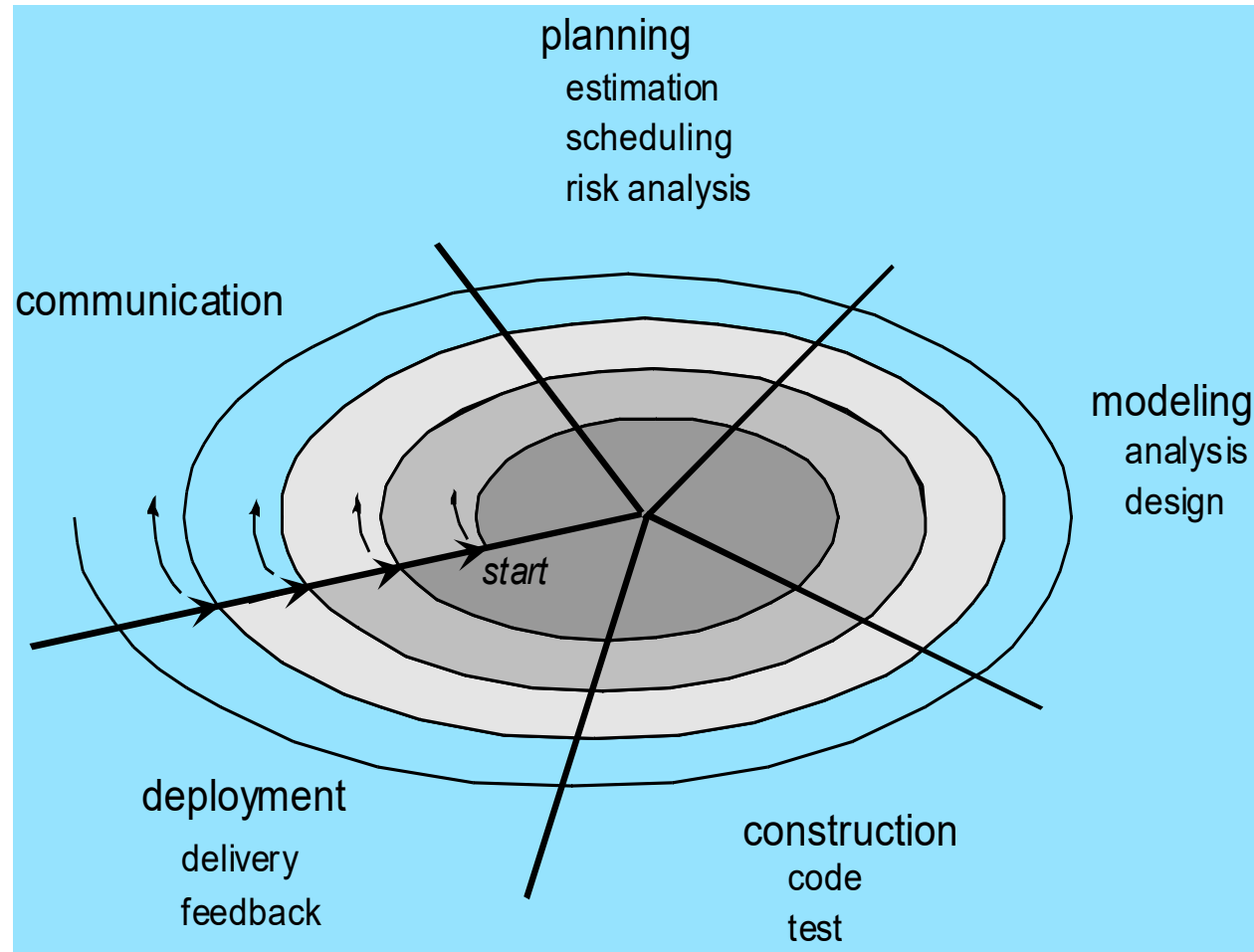
- Prototype should focus on areas of the product that are not well-understood;
- Error checking and recovery may not be included in the prototype;
- Focus on functional rather than non-functional requirements such as reliability and security

Throw-away prototypes



- ✧ Prototypes should be discarded after development as they are not a good basis for a production system:
- It may be impossible to tune the system to meet non-functional requirements;
 - Prototypes are normally undocumented;
 - The prototype structure is usually degraded through rapid change;
 - The prototype probably will not meet normal organisational quality standards.

Evolutionary Models: The Spiral



Other Process Models



- ✧ **Component-based development**—the process to apply **when reuse is a development objective**
- ✧ **Formal methods**—emphasize the **mathematical specification of requirements**
- ✧ **Aspect-Oriented Software Development (AOSD)**—**provides a process and methodological approach** for defining, specifying, designing, and constructing *aspects*
- ✧ **Unified Process**—a “**use-case driven, architecture-centric, iterative and incremental**” software process closely aligned with the Unified Modeling Language (UML)



Model Questions and Guide!

Scenario Question - 01



Case Study 1a: Imagine that a startup company, QuickHealth, is developing a mobile application to provide users with personalized health advice based on their diet and exercise routines. The company wants to release the core features quickly and then add more advanced features, such as fitness device integration, in subsequent releases.

Case Study 1b: A video game development company, GameInnovate, is exploring a new game concept that involves augmented reality (AR). Since this technology is new to the team, they want to understand user interactions and preferences before fully committing to the game's development.

Case Study 1c: Consider a government project, GovtPrj to develop a secure online voting system. The system must be highly secure, reliable, and user-friendly. Given the high risks and complex requirements, the project team needs to ensure that each aspect of the system is thoroughly evaluated and tested.

- For each of the case studies, evaluate the suitability of the **Spiral Model**, **Incremental Process Model**, and the **Prototyping Model**, considering the specific requirements and constraints of developing the projects.

Scenario – 1a



○ **Incremental Process Model – Most Suitable**

The company wants:

- Rapid release of core features quickly
 - Gradual enhancement through future updates
 - Integration with fitness devices in later stages
-
- The **Incremental Model** delivers the system in **small, functional releases**, ensuring:
 - Quick deployment of **core features**
 - Progressive integration of **advanced functionalities**
 - Allows early **market entry and feedback**
 - Supports **continuous feature expansion**
 - Reduces **time-to-market**
 - Facilitates **continuous testing and improvement**
 - **Discuss some of these features here.!!!!**

Scenario – 1a Cont.



- The product is divided into builds (increments). The first increment can contain the core features (diet/exercise logging, basic advice), allowing QuickHealth to establish a market presence and generate revenue quickly. Subsequent increments can deliver the more complex features (wearable integration, advanced analytics) based on user feedback from earlier releases. This aligns perfectly with the "release quickly then add features" requirement.

Scenario – 1b



- GamelInnovate Augmented Reality Game - **Prototyping Model – Most Suitable**

GamelInnovate is developing a **novel AR-based game**, but:

- The technology is **new to the team**
- User behaviour and expectations are **uncertain**
- The company wants to **explore feasibility before full-scale development**
- **Etc.**

Prototyping is ideal for:

- **List and discuss the features of the Prototype Model here.....!**

Scenario – 1b Cont.



- The goal is not to build the final game, but to understand how users interact with AR. GameInnovate can build several "throw-away" or "evolutionary" prototypes focusing on specific interactions (e.g., moving in AR, interacting with virtual objects). User feedback on these prototypes will directly answer their questions about preferences and usability, allowing them to decide if and how to proceed with full development.

Scenario – 1c



- Similarly!



Question 2

Scenario Question - 02



- TechHealth Inc. is developing a new health monitoring application that integrates with wearable devices to track vital signs and provide health recommendations. The project is in its early stages, and the requirements are not fully defined. Given the uncertain and evolving nature of the project requirements:
- Evaluate **the suitability** of the Plan-driven process (The Waterfall Model) for TechHealth Inc.'s development of the health monitoring application and discuss the **potential challenges** and **benefits** of using the Waterfall Model in this scenario.

Answer to Que. 2 Begins



- This analysis evaluates the **suitability, benefits, and challenges** of applying the Waterfall Model to TechHealth Inc.'s project.

Suitability Evaluation



- Given that **project requirements are uncertain and evolving**, the Waterfall Model is **largely unsuitable** for this development scenario.
- The Waterfall Model is most effective when requirements are **well-understood, stable, and unlikely to change**. In contrast, TechHealth's project involves **integration** with wearable devices, evolving health standards, and **user feedback** that will likely emerge only after initial releases. The **lack of fully defined requirements** makes Waterfall a high-risk choice, as it assumes upfront completeness and discourages iterative refinement.

Potential Benefits of Using Waterfall



- Despite the mismatch, there are some potential advantages:
- **Advantages of the Waterfall Model should be discussed in relation to the problem.....!**

Potential Challenges of Using the Waterfall Model



- The evolving nature of the project introduces several critical problems:
- **Disadvantages of the Waterfall Model should be discussed in relation to the problem.....!**